

Manuscript revised August 13, 2026.

Digital Object Identifier Not assigned

XATF: A Carrier-Binding Verification Profile with Typed Failure Attribution for AI-Agent Evidence over X-Road

ANTON SOKOLOV

Tyche Institute, Tallinn, Estonia (e-mail: anton@tyche.institute)

Corresponding author: Anton Sokolov (e-mail: anton@tyche.institute).

No external funding was used for this work.

ABSTRACT AI agents increasingly initiate actions across organizational boundaries, but carrier infrastructures usually identify organizations, services, and systems rather than the autonomous actor behind a request. This applied research article asks which carrier fields, evidence claims, and verifier invariants are required to bind agent-specific evidence to an X-Road exchange without making X-Road the AI-agent verifier itself. The main contribution is XATF, an executable carrier-binding verification profile with typed failure attribution, positioned as a research layering pattern rather than a standard or compliance mechanism. XATF specifies Algorithm 1, which evaluates carrier acceptance, carrier/evidence binding, and Agent Evidence Package (AEP) appraisal under an ordered invariant taxonomy. The evaluation combines a real public X-Road development-stack exchange across two Security Server containers, a 10-request carrier matrix, a signed seven-case corpus with Python/Node verifier parity, and a new 18-case strict-profile red-test run. The real X-Road run completed an HTTP 200 exchange, carried an AEP hash, and reconciled that hash with X-Road request identifiers and message-log rows on both sides. The carrier matrix showed that X-Road rejects transport faults while still carrying verifier-negative requests whose inner evidence fails offline appraisal. The strict-profile run matched 18/18 expected outcomes and 18/18 expected attributions in both Python and Node. A signature-only baseline accepted 11 strict-negative cases that XATF rejects, including a carrier-binding mismatch with valid signatures. Measured timing over 50 warmup and 1000 measured sequences produced median 18.296 ms accept-path and 18.316 ms reject-path verification. Request sizes show the carriage trade-off: 289 bytes for hash-only, 352 bytes for URI-plus-hash, and 23,360 bytes for full-AEP base64. The result is a scoped systems contribution: X-Road remains the outer PKI-backed carrier and audit witness, while AEP supplies agent identity, delegation, policy, runtime, freshness, revocation, and cryptographic evidence for independent verification aligned with RATS, EAT, COSE, and adjacent evidence-supply-chain standards.

INDEX TERMS Agent Evidence Package, Agent Trust Framework, AI agents, digital public infrastructure, Entity Attestation Token, ML-DSA, PKI governance, RATS, Security Server, X-Road.

I. PROBLEM AND RESEARCH QUESTIONS

The problem this article addresses can be stated without any acronym. A message can be perfectly signed and perfectly delivered, and still carry evidence that describes a different request than the one that actually happened. No signature check can catch this, because every signature is valid. What is missing is a check that the evidence inside the message binds to the specific exchange the carrier recorded, performed by a verifier that can say so offline, after the event, from preserved records. Sections VI-A and X make this concrete as case AEP-03: valid signatures over evidence that names the wrong carrier request.

X-Road is a mature interoperability and trust layer for organizations, subsystems, Security Servers, and service calls [1]–[3]. Its trust model is built around organizational membership, service addressing, Security Server mediation, Public Key Infrastructure (PKI), Online Certificate Status Protocol (OCSP), timestamping, global configuration, and message logging. These are valuable properties for public-sector and regulated data exchange.

AI-agent accountability asks an additional class of questions: which agent instance caused the action; under whose authority it acted; which policy and tool manifest constrained it; whether the requested action was inside delegated scope;

whether revocation, replay, tampering, and confused-deputy behavior can be detected after the event; and whether an auditor can bind those answers to the actual carrier exchange.

The research problem is therefore not whether another attestation token can be signed. It is whether a governed carrier can provide enough evidence to let an independent verifier distinguish three facts that are often collapsed in audit prose: the carrier accepted the exchange, the carried evidence binds to that exchange, and the agent evidence itself satisfies a profile.

The study evaluates three research questions:

- **RQ1: carrier evidence sufficiency.** Which X-Road carrier fields are sufficient to reconstruct an offline binding between a message-log record, request payload, and AEP hash?
- **RQ2: typed failure attribution.** Can a carrier-bound verifier return deterministic typed rejection labels across transport, binding, cryptographic, freshness, replay, schema, key, and policy invariants?
- **RQ3: strict-profile cost and closure.** What mandatory invariant families are required to close a strict AEP verifier profile, and what timing and carriage cost do they introduce?

This framing avoids an overclaim. X-Road need not become an AI governance engine. It can remain the carrier and witness. The agent-specific claims can live in an inner evidence object that is independently signed, hashed, and verified.

The scope of every claim is fixed before any result is reported: Section VII opens with the threat model, including its exclusions: compromised signing keys, compromised Security Server hosts, rewritten message logs, and verifier implementation bugs are all outside it. The study tests boundary evidence under that stated model, not a hostile takeover of the stack.

II. TECHNICAL CONTRIBUTIONS

The revised engineering contribution is the X-Road Agent Trust Fabric (XATF) verification profile and its public lab evaluation. One framing sentence governs everything that follows: XATF is a research layering pattern with an executable verifier profile. It is not a standard, a specification, a product, a compliance mechanism, or an independently validated trust framework, and no such status is claimed. Its contribution is a carrier-binding verification profile with typed failure attribution and a systems integration, not a new cryptographic primitive.

Contribution	Technical content
Verifier profile and algorithm	Formal decision relation $\text{accept} := C$ and B and E ; Algorithm 1 with ordered invariant taxonomy, soundness, determinism, canonical claims, signatures, policy/tool/runtime digests, freshness, replay, trusted keys, and carrier binding.
Reconciliation algorithm	Offline procedure that consumes X-Road request/log fields, payload hashes, AEP bytes, signatures, and policy inputs, then returns a typed attribution rather than a single generic failure.
Real carrier evidence	Two public development Security Servers with scripts, logs, hashes, response artifacts, message-log rows, and an HTTP 200 exchange carrying an AEP hash.
Carrier matrix	Ten real X-Road requests covering hash-only, full-AEP, URI-plus-hash, transport-negative, and verifier-negative cases.
Strict red-test corpus	Eighteen v0.6 AEP cases with 18/18 expected outcomes and 18/18 attribution matches in Python and Node after hardening the v0.5 negative-control gaps.
Baseline and measurement	Signature-only baseline, request-size, package-size, and median/IQR offline-verification measurements that expose the engineering trade-off between hash-only, URI-plus-hash, and full-AEP carriage.

III. BACKGROUND

A. X-ROAD AS CARRIER INFRASTRUCTURE

X-Road uses Security Servers and governed configuration to mediate service calls between members and subsystems [2]. Its protocol and transport documentation define message and request structures [4], [5]. Its message-log data model is especially relevant to this study because auditability depends on more than a successful HTTP response [6].

X-Road is treated as the outer carrier. It can identify the consumer subsystem, provider subsystem, service, Security Server path, request identifiers, and message-log records. It cannot, by itself, prove which AI agent instance acted or whether that agent had current delegated authority.

B. REMOTE ATTESTATION, EAT, AND AEP

The Remote Attestation procedureS (RATS) architecture defines a general model for evidence, endorsements, appraisal, and attestation results [7]. The Entity Attestation Token (EAT) provides a token format for attestation claims [8]. Recent IETF Internet-Draft work, not ratified in the same sense as RFCs, extends the same neighborhood toward autonomous-agent EAT profiles, agent authentication and authorization, capability attestation for agentic systems, and measured components [9]–[12]. JSON Canonicalization Scheme (JCS), CBOR Object Signing and Encryption (COSE), CBOR Web Token (CWT), timestamping, Public-Key Infrastructure using X.509 (PKIX), and OCSP provide important building blocks for portable evidence [13]–[18].

An Agent Evidence Package (AEP) is used as the inner evidence object. In this lab, AEP follows EATF, the Agent Trust Framework, an open Tyche Institute reference implementation and documentation set for offline-verifiable agent evidence [19]–[21]. The EATF/AEP material is an open de-

sign artifact, not a ratified standard or trust-service scheme. AEP packages claims about the agent, task, delegation, policy, runtime, payload digest, transport context, freshness, and revocation state. A production profile should be represented using a standards-compatible EAT/COSE design where feasible.

C. WHAT IS NEW HERE

Each adjacent family already solves part of the problem, and the delta is the same in every row: none of them binds the inner evidence to the specific carrier request record and reconciles the two offline with a typed statement of what failed.

Prior art	What it provides	What it does not provide
Plain signed tokens (JWT/JWS-style claims)	Integrity and authenticity of the claim bytes	No binding of those bytes to a specific carrier exchange; no typed attribution
EAT / RATS [7], [8]	Attestation claim format and appraisal architecture	No governed-carrier reconciliation procedure; carrier fields are out of scope
COSE / CWT [14], [15]	Portable signing and token envelopes	Envelope only; no carrier record, no failure taxonomy
X-Road message logging [6]	Governed carrier records on both exchange sides	No appraisal of inner agent evidence; a logged HTTP 200 says nothing about delegation, freshness, or policy
Supply-chain attestation (in-toto, SLSA, SCITT) [22]–[24]	Provenance and transparency for artifacts and build steps	Binds evidence to artifacts and pipelines, not to a per-request carrier exchange record

The delta is measured, not asserted. Over the eighteen strict-profile packages of Section X-B, a signature-only checker (the first row of the table, applied honestly) accepts 11 of the 13 cases the strict profile rejects, including the carrier-binding mismatch in which every signature is valid. That gap is what the rest of this article constructs, executes, and measures.

D. EATF CONTEXT

The Agent Trust Framework (EATF) is the project-origin architecture and reference-implementation context for agent evidence and offline verification. Because EATF materials are self-published by the same research ecosystem, they are treated as design inputs and implementation context rather than independent validation.

The EATF AEP profile motivates a ZIP-based evidence package containing canonical bytes, hash, metadata, receipt, public key, signatures, and timestamp material. The crypto case run exercises the same core idea: hash, sign, timestamp, and verify an evidence package offline.

E. GOVERNANCE MOTIVATION

The European Union Artificial Intelligence Act (EU AI Act) and eIDAS 2.0, the EU electronic-identification and trust-services regime, create policy pressure for accountable, traceable, identity-aware digital processes [25]–[29]. The United

States National Institute of Standards and Technology Artificial Intelligence Risk Management Framework (NIST AI RMF) similarly provides a vocabulary for trustworthy AI risk management [30]. No legal-compliance claim is made. The study asks whether existing trust infrastructure can support evidence patterns that make compliance discussions technically inspectable.

F. PKI AS GOVERNANCE INFRASTRUCTURE

PKI is treated as governance infrastructure: a way of allocating authority, membership, revocation, time, policy, auditability, and relying-party meaning through cryptographic artifacts. X.509 and RFC 5280 supply the certificate-path grammar; RFC 3647 supplies the certificate-policy and certification-practice vocabulary; OCSP and trusted timestamps add status and time evidence [16]–[18], [31], [32].¹

European trust-service regulation and European Telecommunications Standards Institute Electronic Signatures and Infrastructures (ETSI ESI) standards show the same pattern in institutional form: cryptographic trust becomes operational governance through policy, audit, validation, qualified trust services, and trusted lists [27], [28], [33]–[36]. WebPKI offers a useful contrast through Certification Authority/Browser Forum (CA/Browser Forum) rules and Certificate Transparency [37]–[39]. It is more open and browser-mediated than X-Road, but it makes the same uncomfortable point: certificates are never just keys; they are decisions about who may be relied upon, for what, under which rules.²

For AI-agent evidence, this matters because the hard question is not whether a package can be signed. The hard question is whether the surrounding governance regime can answer who issued the authority, what was delegated, when status was checked, what was logged, and how later auditors should interpret the result. Post-quantum migration work, including Module-Lattice-Based Digital Signature Algorithm (ML-DSA) and the broader NIST post-quantum cryptography (PQC) family, makes this more urgent because evidence may need to outlive today's algorithm choices [40]–[43].

IV. DATA BOUNDARY

The study uses public X-Road tooling, Docker-based development containers, and synthetic evidence. The single boundary is that the evidence is public-lab material, not production deployment material.

This boundary is part of the method. A public reproducible lab is stronger for external review than a private demonstration whose important facts cannot be inspected.

V. ARCHITECTURE PATTERN: XATF

The proposed pattern is X-Road Agent Trust Fabric (XATF):

¹RFC 5280 gives certificate-path grammar and validation inputs. It does not make any issuer wise, any policy humane, or any relying party awake. Governance remains the expensive part.

²WebPKI is a comparison, not a template. X-Road is a bounded institutional fabric; WebPKI is a large public trust market with browser enforcement and several battle scars.

Step	Carrier fabric	Agent-evidence layer
1	Security Server mediates the request.	Agent creates signed evidence claims.
2	Message log records request identifiers and hashes.	AEP binds agent, policy, payload, freshness, and delegation.
3	Auditor extracts carrier record.	Verifier appraises signatures, scope, revocation, and replay.
4	Carrier result explains transport status.	AEP result explains agent-authority status.

FIGURE 1. Conceptual XATF evidence flow.

Layer	Function	Evidence
X-Road carrier	Organizational trust, subsystem addressing, Security Server mediation, PKI, message logs	X-Road request ID, message ID, request hash, service identifiers, Security Server logs
AEP inner evidence	Agent identity, delegation, policy, runtime, payload/context binding, freshness, revocation	Signed package, canonical bytes, hash, signatures, timestamp, policy claims
Offline verifier	Reconciliation and appraisal	Hash checks, signature checks, carrier binding checks, policy checks, freshness/replay checks

The intended evidence chain is:

Stage	Binding material
Carrier	X-Road request and log identifiers
Payload	Request payload hash
Evidence	AEP hash, signatures, and claims
Decision	Offline verifier result

This division preserves X-Road's role. X-Road does not need to parse or standardize every AI-agent evidence field. It needs to preserve enough carrier evidence that a verifier can bind the agent evidence to the real exchange.

A. DESIGN PRINCIPLES FOR THE PATTERN

Let C mean that the X-Road carrier accepted and recorded the exchange, E mean that the AEP verifier accepted the agent evidence, and B mean that carrier identifiers and AEP claims reconcile to the same event. The experiment is built around a deliberately simple relation:

$$C = 1 \not\Rightarrow E = 1, \quad \text{accept} := C \wedge B \wedge E.$$

Design principle 1: carrier success is not agent authorization. A successful X-Road exchange can still carry stale, revoked, out-of-scope, or tampered agent evidence. This is not a weakness of X-Road; it is the reason an agent-evidence verifier exists.

Design principle 2: binding is the bridge, not decoration. A positive audit conclusion requires the carrier record,

the binding check, and the evidence appraisal to agree. When they do not agree, the tuple (C, E, B) localizes the problem: transport, evidence, or reconciliation.

VI. XATF VERIFICATION PROFILE

The XATF verifier consumes four classes of input: the X-Road carrier record, the request payload digest, the AEP bytes, and the local verification policy. It returns both a Boolean result and an attribution label. This is the key technical move: a rejection should say whether the failed invariant belongs to transport, binding, cryptographic integrity, freshness, replay, policy integrity, toolchain integrity, schema profile, crypto policy, key status, or agent policy.

Input	Required fields
Carrier record	X-Road ID, X-Road request ID, request hash, consumer/provider subsystem, service, response-body hash, message-log rows.
Payload record	SHA-256 digest of the request payload that carried or referenced the AEP.
AEP package	Canonical claim bytes, stored hash, metadata, OVERT receipt, RSA-4096 signature, ML-DSA-65 signature, public-key material, and timestamp response.
Verifier policy	Accepted profile URI, signature-suite allow-list, trusted policy digest, trusted tool-manifest digest, trusted runtime digest, trusted public keys, freshness window, verification instant t_v , and replay index.

The strict v0.6 profile enforces the following invariants:

Invariant family	Mandatory check
Schema profile	Required claims are present and have the expected types, including <code>package_id</code> , <code>agent_id</code> , <code>delegation_scope</code> , <code>action</code> , <code>issued_at</code> , <code>expires_at</code> , <code>nonce</code> , <code>payload_digest</code> , and <code>xroad_binding</code> .
Canonicalization	Re-serializing the parsed claims under deterministic JSON canonicalization yields exactly the stored <code>canonical.bin</code> bytes.
Cryptographic integrity	The stored SHA-256 hash matches canonical bytes, and RSA-4096 plus ML-DSA-65 signatures verify over those bytes.
Crypto policy and keys	The metadata signature-suite set equals the allow-list, and package public keys match trusted verifier anchors.
Policy/tool/runtime integrity	Claim digests match trusted verifier digests for the policy, tool manifest, and runtime profile.
Carrier binding	AEP carrier fields equal the X-Road ID, request ID, request hash, participants, service, request-payload hash, response hash, and echoed AEP hash from the carrier record.
Freshness and replay	Issue/expiry times fall inside the verifier window relative to the stated verification instant t_v , and package IDs/nonces have not been seen before. Freshness has no meaning except relative to a stated instant; t_v is therefore profile input, not an implicit clock read.
Agent policy	Delegation and revocation states are active, and the action is inside delegated scope.

A. CARRIER-BINDING CORE MECHANISM

The core XATF mechanism is the carrier-binding predicate B . Let R be the X-Road carrier record and P the parsed AEP claim set. In the strict profile:

$$B := \bigwedge_{f \in F} P.\text{xroad_binding}[f] = R[f],$$

where F contains the run verdict, consumer subsystem, provider subsystem, service, X-Road ID, X-Road request ID, X-Road request hash, request-payload hash, response-body hash, and echoed AEP hash. This predicate is intentionally orthogonal to signature validity. A package can have valid RSA-4096 and ML-DSA-65 signatures over its canonical bytes and still fail because those canonical bytes bind to the wrong carrier event. The seven-case corpus demonstrates this with AEP-03; the strict-profile run repeats it as V6-AEP-18. The carrier matrix also records NEG06, where X-Road returned HTTP 200 for a verifier-negative request while offline appraisal rejected the replayed X-Road ID as a freshness/binding failure.

Algorithm 1: XATF.verify(R, d, P, P_i)

Input	X-Road carrier record R , payload digest d , AEP package P , verifier policy P_i .
Output	(decision, label) where decision is PASS or FAIL, and label is one of the ordered invariant families or none.
1	If R records a transport failure, return (FAIL, transport-failure) before appraising AEP evidence.
2	Parse P ; if mandatory entries or claim types are missing, return (FAIL, schema-profile).
3	Recompute canonical bytes and stored SHA-256 hash; on mismatch, return (FAIL, cryptographic-integrity).
4	Verify RSA-4096 and ML-DSA-65 signatures and enforce the signature-suite allow-list; on violation, return the crypto-integrity or crypto-policy label.
5	Check trusted public keys plus policy, tool-manifest, and runtime digests; return key-status, policy-integrity, or toolchain-integrity on violation.
6	Evaluate carrier-binding predicate B against R, d , and P ; on mismatch, return (FAIL, carrier-binding).
7	Check freshness, package-ID replay, nonce replay, delegation status, revocation status, and delegated scope; return the first violated freshness, replay, or agent-policy label under order O .
8	Return (PASS, none).

Proposition 1: soundness under the profile. If Algorithm 1 returns PASS, then every mandatory invariant in the strict XATF profile held for (R, d, P, P_i) .

Proposition 2: deterministic attribution. If one or more mandatory invariants fail, Algorithm 1 returns the label of the first violated invariant family under the total order O . The result is therefore deterministic for both single-fault and multi-fault packages. The verifier cost is $O(|O|)$ invariant checks plus two signature verifications over canonical bytes.

First-failure attribution is a reporting convention, not a detection limit, and the choice deserves its justification. Reporting the first violated family under a fixed order yields an attribution that is deterministic, stable across runs, and therefore comparable between implementations (the property the parity check in Section XI depends on), and it matches operator triage, where the first blocking cause is fixed before the next is visible. The trade-off is diagnostic completeness: a multi-fault package carries one label. The implementations record the complete failure list in the per-case verifier logs (the `failures` field), so the full violation set remains available; only the attribution label is single-valued.

The invariant families themselves carry a deliberately bounded claim. They partition the failure classes observed in this corpus (schema, canonicalization, cryptographic integrity, crypto policy and keys, policy/tool/runtime integrity, carrier binding, freshness and replay, and agent policy), and they are sufficient to separate those observed classes with distinct labels. They are not claimed to be complete against all future failure modes, and no such completeness argument is attempted.

The profile is intentionally stricter than the earlier red-test verifier. A permissive verifier can show that some valid packages pass, but it cannot support an IEEE-style systems claim. The strict profile turns each negative-control label into a concrete checked invariant.

VII. METHOD

The lab uses a modest threat model. The adversary may tamper with agent-evidence bytes, replay a package, present stale delegation, mismatch carrier identifiers, mutate or hide an evidence artifact, or send a request that the carrier should reject. The adversary is not assumed to control the X-Road Security Server host, forge the lab CA or TSA keys, rewrite already exported message-log rows, or compromise the verifier implementation itself. In short, the study tests boundary evidence, not a hostile takeover of the whole stack. Verifier implementation bugs are explicitly outside the threat model; the evaluation assumes the Python and Node implementations correctly encode the stated profile and treats independent security review as future work.

Threat model summary

Adversary move	Expected detector	Out of scope
Tamper with package bytes, signatures, payload hash, or canonical claims	AEP hash, signature, and canonicalization checks	Compromised signing keys
Replay, stale timestamp, revoked delegation, or out-of-scope action	AEP freshness, revocation, delegation, and policy checks	Legal authority to delegate
Carrier mismatch, unreachable artifact, invalid client, or invalid service	X-Road transport result plus offline carrier-binding verifier	X-Road conformance certification

Within that boundary, the method has nine steps.

First, a dual-envelope lab generated payloads, carrier records, AEP-like evidence packages, trust-list data, and offline verifier logs. The lab tests failure attribution: carrier-only, agent-only, both-layer, degraded-carrier, and no-failure cases.

Second, one valid evidence package hash was promoted into a real public X-Road development-stack exchange. The request carried the AEP hash and produced real X-Road response headers and message-log rows.

Third, a separate EATF/AEP crypto case run generated `.aep` packages bound to the real X-Road exchange. These packages used SHA-256 over canonical JSON, RSA-

4096/SHA-256 signatures, ML-DSA-65 signatures, and local RFC 3161 timestamp responses.

Fourth, the AEP cases were assembled into a reusable Evidence Corpus. The corpus records a machine-readable manifest, JSON schemas, AEP ZIP artifacts, JCS-compatible JSON projections, experimental EAT/COSE JSON projections, corpus-level RSA-4096 and ML-DSA-65 signatures, and parity results from independent Python and Node verifiers.

Fifth, the two-Security-Server lab was reused for a real carrier-mode matrix. The matrix sent hash-only, full-AEP base64, and URI-plus-hash requests through X-Road, then sent transport-negative and verifier-negative requests to separate carrier acceptance from agent-evidence appraisal.

Sixth, an initial v0.5 negative-control expansion was run to expose which expected-negative packages the permissive verifier still accepted. This was treated as red-test evidence, not as a success result.

Seventh, a strict v0.6 verifier profile was implemented as an explicit set of mandatory invariants. It generated and checked eighteen AEP packages with real mutations for policy-digest mismatch, toolchain-digest mismatch, transcript/hash breakage, expired and future timestamps, replayed package identifiers, malformed claim types, missing mandatory claims, unsupported algorithms, noncanonical bytes, invalid signatures, untrusted keys, and X-Road request-ID mismatch.

The v0.6 package generator and Python verifier share the same profile contract, so the strict result is also checked by a separate Node verifier over the generated package index. A signature-only baseline is run over the same packages to show what is added beyond ordinary hash/signature validation.

Eighth, the manuscript and reproducibility reports were built only from recorded artifacts: scripts, hashes, logs, message IDs, verifier outcomes, request sizes, package sizes, and verification timings.

Ninth, the public bundle adds review scaffolding around the evidence rather than silently inventing more data: a standards-reference atlas, claim-evidence matrix, fixture backlog, public bundle manifest, URL-check report, and review cards. This extension is therefore a reproducibility and submission-readiness hardening pass. Additional carrier-boundary cases such as certificate rotation, OCSF outage, timestamp outage, stale global configuration, and payload-size bands are outside the reported result set and are treated only as scope boundaries for later independent runs.

VIII. CARRIER-AGENT CASES

The lab generated the following scenario families:

Scenario family	Observed result	Meaning
X-Road-only baseline	Carrier passes, agent evidence absent	Organizational trust alone does not answer agent accountability questions.
Valid attested agent	Carrier passes, agent passes	Dual-envelope pattern is viable.
Hash metadata/full body/attachment-style carriage	Carrier passes, agent passes	Multiple AEP carriage modes can be modeled.
Payload tampering	Carrier and agent fail	Cross-layer payload binding detects tampering.
Evidence tampering	Carrier passes, agent fails	Healthy carrier does not imply valid agent evidence.
Replay/stale nonce	Carrier passes, agent fails	Freshness belongs to the AEP verifier.
Revoked or suspended delegation	Carrier passes, agent fails	X-Road membership does not imply current agent authority.
Invalid certificate/OCSF	Carrier fails, agent can remain internally valid	Carrier PKI and inner evidence validity are separable.
TSA/global-configuration degradation	Carrier degrades, agent passes	Long-term proof and freshness status must be explicit.
Confused-deputy action	Carrier passes, agent fails	Delegated scope needs agent-layer appraisal.

The table groups executed cases into scenario families rather than one row per run. The visible carriage row covers three valid AEP carriage modes, while carrier-degradation labels group closely related certificate/OCSF and timestamp/global-configuration conditions; the remaining visible rows are single-case families.

The verifier completed all expected cases. The three valid attested-agent carriage variants reached 7/7 audit completeness for actor, authority, runtime, policy, payload binding, timestamp, and transport-context fields. The X-Road-only baseline scored 0/7 for agent-specific accountability.

IX. REAL X-ROAD CARRIER RUN

The real run used the public X-Road repository and public development images:

Input	Display pin
X-Road source commit	3c37ca1e...db254081
Central Server image	xrddev-central-server@sha256:1041e967...e7c66c
Test CA image	xrddev-testca@sha256:018e9f6e...16c0c5
Security Server image	xrddev-security-server@sha256:6100f3aa...e68576

Full image digests and source hashes are recorded in the reproducibility reports and checksum files; the table shows shortened display pins to keep the manuscript readable.³

The public dev-image scenario required compatibility fixes: raw octet-stream uploads for anchors and token certificates, explicit signer/proxy Java Management Extensions (JMX) ports, use of the available software token, and a proxy restart after authentication certificate activation. These are recorded in the blocker and reproducibility reports included with the supplementary package.

The successful exchange:

Field	Value
Verdict	REAL-XROAD-CLEAR
Consumer	DEV/COM/4321/TestClient on SS1
Provider	DEV/COM/1234/TestService on SS0
Service	DEV/COM/1234/TestService/mock1
Development proxy endpoint	Recorded in run artifacts
HTTP status	200
X-Road ID	tyche-real-run-20260527T111419Z
X-Road request ID	679b103a...a71e6
AEP hash	f144e683...ce96d0

The request payload hash is recorded in shortened form in the binding summary below. The X-Road response carried an `x-road-request-hash` header; the full Base64 value is recorded in the response-header artifact rather than printed across the manuscript margin.

The provider echoed the same AEP hash. Message-log exports from both Security Servers contained the same carrier identifiers:

Security Server	Rows
SS0 provider	messagelog.logrecord ids 9 and 10
SS1 consumer	messagelog.logrecord ids 4 and 5

The resulting binding can be summarized without printing the full artifact values in the column margin:

Request ID	679b103a...a71e6
Message ID	tyche-real-run-20260527T111419Z
Payload SHA-256	822d59bc...e4cdac6b
AEP SHA-256	f144e683...ce96d0

³Shortened identifiers are for typography only. The full values remain in the recorded artifacts, where they can be copied, hashed, and rechecked without trusting a line break in a PDF.

X. EATF/AEP HYBRID CRYPTO CASES

The EATF/AEP extension generated seven `.aep` packages in the AEP package directory. Each package was bound to the real X-Road exchange and verified offline. The OpenSSL 3.5.0 run exercised SHA-256 digests over canonical JSON evidence, RSA-4096/SHA-256 signatures, ML-DSA-65 signatures aligned with NIST Federal Information Processing Standard (FIPS) 204, local RFC 3161 timestamp responses, carrier-binding checks against `message-binding.json`, and replay, delegation, revocation, scope, and tamper checks [41].

Observed outcomes:

ID	Mutation	Expected	Observed	Attribution
AEP-01	valid hybrid bound	PASS	PASS	none
AEP-02	tampered canonical bytes	FAIL	FAIL	cryptographic integrity
AEP-03	X-Road request ID mismatch	FAIL	FAIL	carrier binding
AEP-04	revoked delegation	FAIL	FAIL	agent policy
AEP-05	replayed nonce	FAIL	FAIL	freshness
AEP-06	action outside scope	FAIL	FAIL	agent policy
AEP-07	ML-DSA signature tamper	FAIL	FAIL	cryptographic integrity

In this table, `FAIL` means successful rejection, not a failed research run. Each negative case changes one property while holding the rest of the package structure still: canonical bytes, carrier request ID, delegation status, freshness nonce, action scope, or ML-DSA signature. That makes the failure useful because the verifier can say which invariant broke.

The contract result was 7/7 expected outcomes matched. The most important case is AEP-03: the AEP signatures are valid, but the X-Road request ID inside the AEP does not match the real carrier record. This demonstrates why carrier binding is a distinct verifier responsibility, not merely a signature check.

A. BOUNDED NEGATIVE-CONTROL EXPANSION

A later bounded fixture expansion generated seventeen candidate `.aep` packages with OpenSSL 3.5.6. Unlike the accepted corpus, this is not presented as a green acceptance corpus. It is a negative-control result: five expected-positive packages passed, but only three of twelve expected-negative packages failed closed. The other negative candidates were accepted, which means either the verifier profile did not yet enforce the intended invariant, or the fixture mutation did not actually break the signed evidence in the way the case name implied.

The expected-positive identifiers were V5-AEP-01, V5-AEP-02, V5-AEP-03, V5-AEP-04, and V5-AEP-17. The expansion used OpenSSL 3.5.6 because it was run as a later hardening pass after the seven-case OpenSSL 3.5.0 corpus; the results are therefore reported separately as negative-control evidence.

Fixture group	Observed result	Publication meaning
Expected-positive package variants (5 cases)	5/5 PASS	Minimal, full, URI-only, detached, and large-payload-reference packages are executable candidate variants.
Negative controls caught (3 cases)	3/12 FAIL	The current verifier detects canonical/signature breakage, future issue time, and one missing-claim/policy interaction.
Negative controls missed (9 cases)	9/12 PASS	Policy-hash, toolchain-hash, replay, unsupported-algorithm, key-status, and related checks need stricter profile enforcement or better fixture mutation.

The failed-closed cases are concrete. V5-AEP-07 changed the transcript hash and was rejected for cryptographic integrity: canonical-hash, RSA, and ML-DSA checks all disagreed. V5-AEP-09 used a future issue time and was rejected for freshness. V5-AEP-12 removed a mandatory claim and was rejected through an agent-policy interaction, which is still useful but not as clean as a dedicated schema-profile rejection.

Negative result	Cases	Verifier-profile lesson
Caught	07, 09, 12	Integrity and freshness checks are active; schema failures need cleaner attribution.
Missed	05, 06, 08, 10, 11, 13, 14, 15, 16	Policy-hash, toolchain-hash, expired timestamp, replay, malformed-claim, unsupported-algorithm, canonicalization, invalid-signature, and unknown-key cases need stricter mandatory checks or stronger fixture construction.

The missed v0.5 cases are interpreted narrowly. Several fixture labels did not force the mutated value into the verifier's signed processing path, and several intended checks were not yet mandatory profile invariants. The red-test result therefore identifies invariant-implementation gaps, not merely fixture defects. It no longer only shows that a few packages can pass; it records where the next verifier profile is too permissive. In research terms, the negative controls convert generic caveats into named verifier tasks.

B. STRICT PROFILE V0.6

The v0.6 rescue pass converts those named verifier tasks into mandatory checks. It generated eighteen new AEP packages under the same public and synthetic boundary, using OpenSSL 3.5.0 and the recorded X-Road carrier binding. Unlike the v0.5 expansion, the v0.6 negative cases contain concrete machine-checkable mutations: wrong trusted digests, noncanonical bytes, invalid signatures, unsupported suites, replayed identifiers, malformed claims, expired or future times, untrusted key material, and X-Road request-ID mismatch.

v0.6 family	Result	Attribution
Five valid package variants	5/5 PASS	none
Policy/toolchain integrity	2/2 FAIL	policy-integrity; toolchain-integrity
Crypto/canonical integrity	3/3 FAIL	cryptographic-integrity
Freshness and replay	3/3 FAIL	freshness; replay
Schema profile, crypto policy, key status	4/4 FAIL	schema-profile; crypto-policy; key-status
Carrier binding	1/1 FAIL	carrier-binding

The v0.6 contract matched 18/18 expected outcomes and 18/18 expected attribution labels in both Python and Node. The timing run used 50 warmup sequences and 1000 measured sequences over the ordered case set. Accept-path verification had median 18.296 ms with IQR 9.386 ms; reject-path verification had median 18.316 ms with IQR 9.296 ms. The generated package sizes averaged 17,587 bytes (16,779–17,971 bytes). These are not production-performance claims, but they make the engineering cost of the strict profile visible and reproducible.

A signature-only baseline over the same eighteen packages accepted eleven strict-negative cases that XATF rejects. Most importantly for the editorial concern, the baseline accepts the carrier-binding mismatch: the signatures and stored hash validate, but the AEP binds to the wrong X-Road request ID. That is the difference between a generic signed-token check and a carrier-bound verifier.

The strict-profile result changes the evidentiary status of the paper. The earlier v0.5 result identified permissive verifier behavior; the v0.6 result demonstrates that the same case families can be represented as enforceable invariants with typed failure attribution.

XI. EVIDENCE CORPUS

The Evidence Corpus turns the crypto cases into a reusable test artifact rather than a one-off run. It contains signed, unsigned, and canonical corpus manifest files; corpus-level RSA-4096/SHA-256 and ML-DSA-65 signatures over the canonical manifest bytes; schema files for the corpus manifest, AEP package, and EAT claims; seven AEP ZIP packages; seven extracted JCS-compatible claims; seven experimental EAT/COSE JSON claim projections; and Python/Node verifier outputs.

The parity result was concise: manifest signature verification passed in Python and Node, expected outcomes matched 7/7 in both implementations, and Python/Node case parity passed.

The v0.6 strict-profile corpus adds a second, machine-checked parity result. Both implementations (the Python verifier and the separate Node verifier, run over the same generated package index and public keys) match the expected contract, and a dedicated parity artifact compares them case by case on three fields: verdict, attribution label, and canonical SHA-256. Across all eighteen cases the comparison records zero differences. The boundary of this claim is stated exactly: dual-implementation parity demonstrates *specification*

agreement, not security assurance. Two implementations that agree can share a misreading of the profile; parity reduces the risk that the result is a property of one implementation path, and independent security review of the verifiers remains necessary and is named as future work.

This matters for the central claim. The evidence package is no longer merely described in prose. It is represented as a signed, hash-addressable, independently verified corpus that can be re-run without trusting the original authorial narrative.

A. REPLAY, REGENERATION, AND CORRECTIONS

The supplementary package ships a replay environment whose base container image is pinned by digest and provides OpenSSL 3.5.7, so a reviewer does not need a host with ML-DSA support; distributions still shipping OpenSSL 3.0 cannot verify these signatures at all. One command replays the shipped corpus: it checks every artifact against its recorded SHA-256, runs both verifiers, and compares them. A second command regenerates the entire corpus from scratch, with new RSA-4096 and ML-DSA-65 key pairs, new timestamps, and new packages, and verifies the result.

Both commands were executed on 2026-08-13, two months after the original run and on a different OpenSSL release. The replay reproduced 18/18 expected outcomes in both implementations with zero parity differences. The from-scratch regeneration matched 18/18 expected outcomes and 18/18 expected attributions, with zero verdict or attribution differences from the original run, while none of the eighteen canonical hashes matched the original bytes, because every key and timestamp was fresh. The signature-only baseline reproduced its count of eleven accepted strict-negatives. This is the direct answer to the reasonable concern that a fixed expected-outcome table might be tuned to a frozen corpus: the outcomes survive complete regeneration of the corpus, so they are a property of the profile, not of the bytes.

Preparing that replay exposed two defects in the originally shipped artifacts, and both are reported here rather than silently repaired. First, the shipped manifest recorded a stale size and hash for one file (the Node parity output), so the supplement failed its own integrity check on exactly that file; the other 51 of 52 entries verified byte-for-byte, the row has been corrected, and the superseded values are preserved in the corrections log. Second, the Python verifier originally evaluated freshness against the wall clock while valid packages carry a ten-minute expiry window, so a replay performed more than ten minutes after generation failed all five expected-positive cases with spurious `freshness` attributions. The verification instant t_v is now an explicit profile input (Section VI), defaulting for a frozen corpus to the corpus issue instant, the same clock the Node verifier already derived from the packages. Neither correction changes any printed number in this article.

XII. REAL CARRIER MATRIX

The carrier-mode matrix exercised ten real requests through the two-Security-Server Docker lab after a clean-room

rebuild and REST continuation. The run stamp was 20260527T145217Z; the development proxy endpoint is recorded in the run artifacts rather than printed in the public manuscript.

The summary result was:

Family	Cases	Transport result	Agent-evidence expectation
Carrier modes	3	3/3 HTTP 200	PASS
Transport negatives	2	2/2 HTTP 500	not reached
Verifier negatives	5	5/5 HTTP 200	FAIL in offline appraisal

The compact case matrix reduces the ten requests to five evidentiary groups. CM01-CM03 returned HTTP 200 and passed AEP appraisal across hash-only, full-AEP, and URI-plus-hash carriage. NEG01-NEG02 returned HTTP 500 before agent evidence was appraised, showing ordinary X-Road transport rejection. NEG03, NEG05, and NEG07 returned HTTP 200 while evidence integrity failed. NEG04 returned HTTP 200 while artifact resolution failed. NEG06 returned HTTP 200 while freshness appraisal failed.

ID	Mutation	Carrier result	Appraisal meaning
CM01	hash-only carriage	HTTP 200	expected AEP PASS
CM02	full-AEP base64 carriage	HTTP 200	expected AEP PASS
CM03	URI-plus-hash carriage	HTTP 200	expected AEP PASS
NEG01	unregistered client	HTTP 500	verifier not reached
NEG02	unknown service	HTTP 500	verifier not reached
NEG03	declared hash mismatch	HTTP 200	integrity FAIL
NEG04	unreachable evidence URI	HTTP 200	artifact-resolution FAIL
NEG05	mutated URI content	HTTP 200	integrity FAIL
NEG06	replayed X-Road ID	HTTP 200	freshness FAIL
NEG07	tampered full AEP	HTTP 200	integrity FAIL

The failed carrier-matrix cases therefore have two meanings. A transport-negative case asks whether X-Road itself refuses an invalid carrier request. A verifier-negative case asks whether X-Road can carry a request whose inner evidence later fails appraisal. The second group is the more interesting one: the carrier did its job, and the verifier did the other job.

Full X-Road IDs, request IDs, request bodies, response headers, and message-log rows are preserved in the carrier-matrix JSONL and CSV tables and in the exported Security Server message logs.

This is the clearest empirical separation in the study. X-Road correctly rejects transport-level problems such as an unknown client or unknown service. It also correctly carries requests whose inner agent evidence is bad: wrong declared AEP hash, unreachable evidence URI, mutated URI content, replayed X-Road ID, or tampered AEP. Those latter cases are not X-Road failures. They are verifier findings.

The matrix exported response headers, response bodies, request payloads, SHA-256 checksums, and message-log CSVs from both Security Servers. The carrier matrix therefore supports the study's central design decision: agent attestation

must be appraised by a verifier layer while X-Road remains the governed carrier and witness.

A. QUANTITATIVE FOOTPRINT

The carrier modes have different transport footprints. Hash-only and URI-plus-hash carriage keep the X-Road payload small and place the full AEP in an evidence store. Full-AEP carriage makes the request self-contained but expands the payload by roughly two orders of magnitude in this lab.

Carrier mode	Request bytes	Engineering meaning
Hash-only	289	Minimal carrier payload; requires separate evidence-package retrieval.
URI-plus-hash	352	Small carrier payload with explicit artifact location and digest binding.
Full-AEP base64	23,360	Self-contained request; much larger carrier payload.

The AEP packages themselves are small enough for ordinary archive and review workflows. The original seven-case hybrid corpus had package sizes of 17, 207–17, 324 bytes; the strict v0.6 corpus, 16, 779–17, 971 bytes. Offline-verification timing for the v0.6 corpus is reported once, in Section X-B. These measurements support a design rule: carry a hash or URI-plus-hash through X-Road for routine exchange, and preserve the full AEP in a retrievable evidence archive unless the workflow requires self-contained transport.

XIII. FINDINGS

A. FINDING 1: X-ROAD CAN CARRY AGENT EVIDENCE WITHOUT BECOMING THE AGENT VERIFIER

The real run demonstrates that a public X-Road development stack can carry an AEP hash and produce carrier identifiers and message-log rows suitable for later reconciliation. This supports X-Road as a carrier and audit witness.

It does not show that X-Road natively verifies agent identity, delegated authority, model/runtime measurements, tool manifests, or agent revocation state. Those checks remain the responsibility of AEP and its verifier.

B. FINDING 2: THE AEP HASH IS A COMPACT BRIDGE BETWEEN LAYERS

The AEP hash is small enough to carry in a request or meta-data field, but strong enough to bind the full agent evidence package to the carrier exchange. The full AEP can remain in an evidence archive, response body, attachment, or out-of-band audit store.

C. FINDING 3: MESSAGE LOGS TURN TRANSPORT INTO WITNESS EVIDENCE

An HTTP 200 response alone is not the evidence. The useful audit object is the reconciliation among X-Road ID, request ID, request hash, payload hash, echoed AEP hash, and message-log rows on both Security Servers.

Witness field	Recorded value
X-Road ID	tyche-real-run-20260527T111419Z
Request ID	679b103a...a71e6
Payload SHA-256	822d59bc...e4cdac6b
AEP SHA-256	f144e683...ce96d0
SS0 provider rows	messagelog.logrecord ids 9 and 10
SS1 consumer rows	messagelog.logrecord ids 4 and 5

D. FINDING 4: NEGATIVE CASES ARE THE ARCHITECTURE

The failure cases show why the layers must remain distinct. A request can travel through X-Road while the agent evidence is revoked, stale, out of scope, or cryptographically tampered. Conversely, agent evidence can remain internally valid while the carrier layer fails.

E. FINDING 5: HYBRID SIGNATURES EXECUTE IN THE RECORDED OPENSLL LAB

OpenSSL 3.5.0 supported ML-DSA-65 in this environment, allowing the AEP case run to use real RSA-4096 and ML-DSA-65 signatures over the same canonical evidence bytes without breaking ordinary PKI validation flows inside the lab. This is not a PQC assurance, scalability, or production deployment claim.

F. FINDING 6: STRICT PROFILES TURN NEGATIVE CONTROLS INTO ENGINEERING REQUIREMENTS

The v0.5 red-test expansion showed that labels alone are not enough: several expected-negative fixtures were still accepted by a permissive verifier. The v0.6 profile closed that gap by requiring concrete invariants for schema shape, canonical bytes, trusted digests, signature suites, public keys, freshness, replay, and carrier binding. The resulting strict-profile contract match and the signature-only baseline (Section X-B) turn that lesson into a checkable engineering requirement rather than a caveat.

XIV. READINESS ASSESSMENT

At the present stage, the carrier-binding result is the strongest part of the package. The lab carried an AEP hash through a real X-Road development-stack exchange, received an HTTP 200 response, and reconciled the echoed hash with the message-binding record, response headers, and SS0/SS1 message-log exports. The offline carrier/agent verifier also has high research readiness: all scenario families completed as expected, and the real carrier-mode matrix executed ten X-Road requests, including eight HTTP 200 outcomes and two expected HTTP 500 transport negatives.

The agent-evidence layer is ready as lab evidence, not as a production profile. Seven RSA-4096/ML-DSA-65 AEP cases matched their expected outcomes, and the signed reusable Evidence Corpus passed Python/Node verifier parity. The initial v0.5 negative-control expansion exposed permissive verifier-profile gaps; the v0.6 strict-profile pass closed them for the reported fixture families (Section X-B).

The remaining missing pieces are explicit. X-Road does not natively identify AI-agent instances; it identifies members, subsystems, services, and requests. The current AEP is

research-profile aligned but not yet a standard EAT/COSE AEP profile. X-Road assurance certification is not claimed. Clean-room reproducibility is partial-to-high for the REST lab: the continuation and carrier matrix completed, but the fresh run required Test CA artifact refresh and proxy restarts.

Remaining gap	Required next action
Binary EAT/COSE profile	Convert the research ZIP/JCS profile into mandatory and optional EAT/COSE claims.
Verifier assurance	Audit the Python and Node verifiers for cryptographic-library misuse and add a third-party implementation.
Production X-Road assurance	Repeat carrier-boundary tests with certificate rotation, OCSP outage, timestamp outage, and stale configuration.
Public data synchronization	Done; the v0.9 reproducibility package is published on Zenodo (DOI 10.5281/zenodo.20613101) under concept record 20468482.

XV. IMPLICATIONS

For Estonian X-Road / X-tee research, the result suggests a public research path: use public development tooling, Docker-based lab setup, test evidence, and open metrics. The value lies in showing how agent evidence can be audited through an existing national-style trust fabric.

For X-Road-like infrastructures more generally, the pattern is to keep the carrier responsible for organizational trust and message accountability; carry or reference an AEP hash in the request/response path; preserve message-log evidence that can be reconciled offline; verify the AEP outside the carrier core; use EAT/RATS/COSE-aligned structures for future standardization; and treat policy, delegation, revocation, and runtime appraisal as agent-layer checks.

A concrete failure scenario is the carrier-binding mismatch. An attacker or confused integration reuses canonical AEP bytes from one X-Road event inside a later request. The signatures and stored hash can still verify, and X-Road can still log an HTTP 200 exchange. XATF rejects the package because the canonical carrier fields no longer match the message-log record. The audit trail then records both facts: the carrier exchange happened, and the agent evidence did not bind to that exchange.

XVI. REFERENCE DISCIPLINE AND POSITIONING

The public package strengthens the argument by widening references horizontally and vertically. Horizontally, the study connects X-Road and AEP to RATS, EAT, COSE, CWT, JCS, C2PA content provenance, Sigstore, in-toto/SLSA software-supply-chain provenance, SPDX, CycloneDX, SCITT, W3C verifiable credentials, and digital-public-infrastructure sources [7], [8], [13]–[15], [22]–[24], [44]–[56].⁴ Vertically, it follows PKI down into certificate policy, revocation, timestamping, trusted-service governance,

⁴C2PA is content provenance, Sigstore and in-toto/SLSA come from software-supply-chain signing and build provenance, SPDX and CycloneDX from software bill-of-materials work, SCITT from transparency-oriented supply-chain integrity, and W3C credentials from portable claims. They are adjacent families, not interchangeable costumes.

WebPKI comparison, and post-quantum migration, including AdES creation/validation procedures and trust-list structure [36], [57].

The reference rule is simple: every source must support a claim, method choice, limitation, or comparison. The bibliography is not a trophy cabinet. It is a map of what the experiment depends on and what it refuses to claim.⁵

The work sits between digital-government infrastructure, security engineering, and applied cryptography. X-Road supplies the governed carrier. PKI supplies certificate-based identity, policy anchors, and audit conventions. AEP supplies agent-specific evidence. Verifiers supply appraisal. Future adjacent work should compare the same pattern across receipt systems, public audit surfaces, selective-disclosure workflows, and evidence observability.

XVII. LIMITATIONS

The carrier run used public development images and a toy provider service. The clean-room rerun exposed development-stack drift: disposable Certification Authority (CA) artifacts had to be refreshed from the current Test CA, Security Server proxies required restart after authentication certificate activation, and the upstream Hurl scenario can fail late in Simple Object Access Protocol (SOAP) access-right setup even when the Representational State Transfer (REST) lab path is usable. The AEP profile is not a ratified standard. The EAT/COSE projection in the Evidence Corpus is an explicit JSON claim mapping, not a binary COSE/CWT token. The EATF materials are self-published project artifacts and must be treated as design context. The lab does not prove AI Act compliance, eIDAS trust-service status, timestamp-service validity, or model safety. The message-log exports are lab database evidence, not an independent legal archive.

These limits bound the claim: carrier-binding feasibility and verifier architecture, not operational certification.

XVIII. FUTURE WORK

Future work should move in four directions. First, the research AEP profile should be converted into an EAT/COSE-compatible profile with mandatory and optional claims. Second, the strict v0.6 verifier should be independently security-reviewed and reimplemented, and checked against binary EAT/COSE, message-log archive, and .aep package validators. Third, carrier-boundary experiments should cover the eight named next tests: certificate rotation, OCSP outage, timestamping outage, stale global configuration, clock skew, larger payloads and carriage-mode comparisons, unavailable evidence stores, and independent verifier audit. Fourth, clean-room reproduction should be stabilized with digest-pinned Compose references, idempotent setup, commands, versions, hashes, expected outcomes, observed outcomes, validator logs, a data dictionary, a threat model, a reviewer checklist, and a one-command runner.

⁵A long reference list is useful only when each source has a job. Otherwise it is just fog with page numbers.

XIX. CONCLUSION

This work specifies and evaluates XATF, a carrier-binding verification profile with typed failure attribution for AI-agent evidence over X-Road. The real X-Road run binds a request ID, message ID, payload hash, and AEP hash across two Security Servers. The EATF/AEP crypto cases show that agent evidence can independently detect tampering, carrier-binding mismatch, revocation, replay, and policy-scope failures. The Evidence Corpus strengthens the result by making those claims reproducible as signed artifacts checked by Python and Node verifier implementations. The carrier matrix adds the decisive empirical split: X-Road rejects transport faults but carries verifier-negative agent evidence, exactly as the XATF layering predicts. The strict v0.6 profile then turns the verifier lesson into an engineering result: eighteen package families match both expected outcome and expected failure attribution under explicit mandatory invariants in both Python and Node, while a signature-only baseline accepts eleven strict-negative cases.

The strongest conclusion is precise: X-Road should not be framed as an AI-agent attestation system. It should be framed as a governed PKI-backed carrier that can support one. AEP supplies the agent-specific evidence. EAT and RATS supply the standards direction. EATF supplies a concrete implementation hypothesis. PKI supplies certificate-based governance controls. XATF binds the pieces into a research program that is technically testable and worth further exploration, especially where AI-agent activity may traverse public-private service exchanges over X-Road-like infrastructure.

The claim is deliberately scoped: the study demonstrates a reproducible public-lab pattern for binding agent evidence to a governed message carrier and appraising that evidence with a strict offline verifier.

DATA AND MATERIALS AVAILABILITY

The data and code package for this article is deposited on Zenodo as version v0.9, record 20613101 [58], under concept record 20468482, which resolves to the latest version. The deposit contains the public bundle manifest, real X-Road run reports, message-log exports, EATF/AEP procedure notes, verifier logs, public keys, test-vector material, corpus manifests and signatures, carrier-matrix CSV/JSONL tables, checksums, negative-control fixtures, claim-evidence matrix, reference policy, and build report. It also includes the strict-profile v0.6 fixture outputs: run report, results CSV, benchmark summary, signature-only baseline, Node parity output, package index, per-case verifier logs, synthetic AEP fixtures, and verifier scripts. The supplementary package accompanying this revision additionally ships the replay environment of Section XI-A: a digest-pinned container definition, a one-command replay that checks artifact hashes and runs both verifiers to a parity diff, a one-command from-scratch regeneration, a data dictionary, the corrections log, and the recorded outputs of the 2026-08-13 replay and regeneration runs. The concept record resolves to the latest deposited version as deposits are updated. Private keys, restricted credentials, and

live operational records are excluded.

PROJECT-ORIGIN AND COMPETING-INTEREST NOTE

The author is affiliated with Tyche Institute. EATF materials cited in the manuscript are Tyche-origin, self-published project artifacts and are used as design context and implementation hypotheses, not as independent validation or as evidence of standardization, certification, legal compliance, or trust-service status. The eAudit.ee service is referenced only as adjacent programme context: it is a production deployment of same-author software operated in an affiliated setting, not an independent adoption or validation claim.

RELATED WORK BY THE AUTHOR AND PRIOR SUBMISSIONS

The thirteen carrier-agent scenarios of Section VIII and the acceptance relation $\text{accept} := C \text{ and } B \text{ and } E$ also appear in the author's workshop paper "Signed, Fresh, and Wrong: Cross-Verifier Disagreement in AI-Agent Evidence over X-Road," accepted at RAISE 2026 (ESORICS 2026 workshops) in July 2026, where they serve as the worked application of a predeclared-oracle attribution protocol; that paper cites the present work as the source of the carrier-binding profile and the full scenario analysis. A conference version of this material was accepted at GAISS 2026 (paper 2727); the author declined registration, no copyright was transferred, and that version is not proceeding to publication. A package-level sibling study of cross-verifier conformance and mutation measurements was submitted to the *Journal of Cybersecurity* and withdrawn at the author's request in July 2026. Reserved to the present article and appearing in no other venue: the XATF verification profile and Algorithm 1; the request-identifier, payload, and AEP binding chain; the real two-Security-Server X-Road run and its message-log reconciliation; typed failure attribution under Proposition 2; the strict-profile v0.6 corpus with dual-implementation parity and its replay/regeneration evidence; the signature-only baseline; and the audit-completeness analysis.

ACKNOWLEDGMENT

Generative-AI assistance. Claude (Anthropic) was used to draft and revise prose throughout the manuscript. All experimental design, all runs, all measurements, and all reported numbers are the author's; no result, log, figure, or citation in this article was generated by an AI system. The author reviewed and verified all AI-assisted text and takes full responsibility for the content. Reported per IEEE, COPE, and ICMJE recommendations.

REFERENCES

- [1] Nordic Institute for Interoperability Solutions, "X-Road Data Exchange," 2026, accessed 2026-05-28. [Online]. Available: <https://x-road.global/data-exchange>
- [2] —, "X-Road Security Architecture," 2026, accessed 2026-05-28. [Online]. Available: https://docs.x-road.global/Architecture/arc-sec_x_road_security_architecture.html
- [3] —, "X-Road Documentation," 2026, accessed 2026-05-28. [Online]. Available: <https://docs.x-road.global/>

- [4] —, “X-Road Message Protocol,” 2026, accessed 2026-05-28. [Online]. Available: https://docs.x-road.global/Protocols/pr-mess_x-road_message_protocol.html
- [5] —, “X-Road Message Transport Protocol,” 2026, accessed 2026-05-28. [Online]. Available: https://docs.x-road.global/Protocols/pr-messtransp_x-road_message_transport_protocol.html
- [6] —, “X-Road Message Log Data Model,” 2026, accessed 2026-05-28. [Online]. Available: https://docs.x-road.global/DataModels/dm-ml_x-road_message_log_data_model.html
- [7] IETF, “Remote Attestation procedureS (RATS) Architecture,” RFC Editor, Tech. Rep. RFC 9334, 2023. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9334.html>
- [8] —, “The Entity Attestation Token (EAT),” RFC Editor, Tech. Rep. RFC 9711, 2025. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9711.html>
- [9] A. Messous, “Entity Attestation Token (EAT) Profile for Autonomous AI Agents,” IETF Internet-Draft, Tech. Rep. draft-messous-eat-ai-01, 2026, work in progress; accessed 2026-06-05. [Online]. Available: <https://datatracker.ietf.org/doc/draft-messous-eat-ai/>
- [10] P. Kasselmann *et al.*, “AI Agent Authentication and Authorization,” IETF Internet-Draft, Tech. Rep. draft-klrc-aiagent-auth-02, 2026, work in progress; accessed 2026-06-05. [Online]. Available: <https://datatracker.ietf.org/doc/draft-klrc-aiagent-auth/>
- [11] K. Huang, “Capability Attestation Extensions for the Entity Attestation Token (EAT) in Agentic AI Systems,” IETF Internet-Draft, Tech. Rep. draft-huang-rats-agentic-eat-cap-attest-00, 2025, expired Internet-Draft; accessed 2026-06-05. [Online]. Available: <https://datatracker.ietf.org/doc/draft-huang-rats-agentic-eat-cap-attest/>
- [12] S. Frost, T. Fossati, H. Tschofenig, and H. Birkholz, “Entity Attestation Token (EAT) Measured Component,” IETF Internet-Draft, Tech. Rep. draft-ietf-rats-eat-measured-component-12, 2026, work in progress; accessed 2026-06-05. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-rats-eat-measured-component/>
- [13] IETF, “JSON Canonicalization Scheme (JCS),” RFC Editor, Tech. Rep. RFC 8785, 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8785.html>
- [14] —, “CBOR Object Signing and Encryption (COSE): Structures and Process,” RFC Editor, Tech. Rep. RFC 9052, 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9052.html>
- [15] —, “CBOR Web Token (CWT),” RFC Editor, Tech. Rep. RFC 8392, 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8392.html>
- [16] —, “Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP),” RFC Editor, Tech. Rep. RFC 3161, 2001. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3161.html>
- [17] —, “Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List Profile,” RFC Editor, Tech. Rep. RFC 5280, 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5280.html>
- [18] —, “X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP,” RFC Editor, Tech. Rep. RFC 6960, 2013. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6960.html>
- [19] Tyche Institute, “EATF Documentation,” 2026, self-published project documentation; accessed 2026-05-28. [Online]. Available: <https://www.eatf.eu/docs>
- [20] —, “EATF Architecture,” 2026, self-published project documentation; accessed 2026-05-28. [Online]. Available: <https://www.eatf.eu/architecture>
- [21] —, “Agent Trust Framework Reference Repository,” 2026, self-published reference implementation; accessed 2026-05-28. [Online]. Available: <https://github.com/tyche-institute/eatf>
- [22] in-toto Project, “in-toto,” 2026, accessed 2026-05-28. [Online]. Available: <https://in-toto.io/>
- [23] SLSA, “Supply-chain Levels for Software Artifacts Specification,” 2026, accessed 2026-05-28. [Online]. Available: <https://slsa.dev/spec/latest/>
- [24] IETF SCITT Working Group, “SCITT Architecture,” IETF Internet-Draft, Tech. Rep., 2026, work in progress; accessed 2026-05-28. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-scitt-architecture>
- [25] European Parliament and Council, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence,” 2024, official EUR-Lex legal anchor; accessed 2026-05-28. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [26] European Commission, “AI Act,” 2026, accessed 2026-05-28. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>
- [27] European Parliament and Council, “Regulation (EU) No 910/2014 on electronic identification and trust services for electronic transactions in the internal market,” 2014, official EUR-Lex legal anchor; accessed 2026-06-05. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2014/910/oj>
- [28] —, “Regulation (EU) 2024/1183 establishing the European Digital Identity Framework,” 2024, official EUR-Lex legal anchor; accessed 2026-05-28. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1183/oj>
- [29] European Commission, “The legal and technical road to EU Digital Identity Wallets,” 2026, accessed 2026-05-28. [Online]. Available: <https://ec.europa.eu/digital-building-blocks/sites/spaces/EUDIGITALIDENTITYWALLET/pages/915931903/The%2Blegal%2Bband%2Btechnical%2BBroad%2Bto%2BEU%2BDigital%2BIdentity%2BWallets>
- [30] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” 2023, accessed 2026-05-28. [Online]. Available: <https://www.nist.gov/itl/ai-risk-management-framework>
- [31] International Telecommunication Union, “ITU-T Recommendation X.509: Information technology - Open Systems Interconnection - The Directory: Public-key and attribute certificate frameworks,” 2025, stable recommendation entry point; accessed 2026-05-28. [Online]. Available: <https://www.itu.int/rec/T-REC-X.509/>
- [32] IETF, “Internet X.509 Public Key Infrastructure Certificate Policy and Certification Practices Framework,” RFC Editor, Tech. Rep. RFC 3647, 2003. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3647.html>
- [33] European Telecommunications Standards Institute, “ETSI EN 319 401 V3.1.1: Electronic Signatures and Infrastructures (ESI); General Policy Requirements for Trust Service Providers,” ETSI, Tech. Rep., 2024, accessed 2026-06-05. [Online]. Available: https://www.etsi.org/deliver/etsi_en/319400_319499/319401/03.01.01_60/en_319401v030101p.pdf
- [34] —, “ETSI EN 319 411-1 V1.5.1: Electronic Signatures and Trust Infrastructures (ESI); Policy and security requirements for Trust Service Providers issuing certificates; Part 1: General requirements,” ETSI, Tech. Rep., 2025, accessed 2026-06-05. [Online]. Available: https://www.etsi.org/deliver/etsi_en/319400_319499/31941101/01.05.01_60/en_31941101v010501p.pdf
- [35] —, “ETSI EN 319 421 V1.3.1: Electronic Signatures and Trust Infrastructures (ESI); Policy and Security Requirements for Trust Service Providers issuing Time-Stamps,” ETSI, Tech. Rep., 2025, accessed 2026-06-05. [Online]. Available: https://www.etsi.org/deliver/etsi_en/319400_319499/319421/01.03.01_30/en_319421v010301v.pdf
- [36] —, “ETSI TS 119 612 V2.4.1: Electronic Signatures and Trust Infrastructures (ESI); Trusted Lists,” ETSI, Tech. Rep., 2025, accessed 2026-06-05. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/119600_119699/119612/02.04.01_60/ts_119612v020401p.pdf
- [37] CA/Browser Forum, “Baseline Requirements for the Issuance and Management of Publicly-Trusted TLS Server Certificates,” 2026, stable document index; accessed 2026-05-28. [Online]. Available: <https://cabforum.org/working-groups/server/baseline-requirements/documents/>
- [38] IETF, “Certificate Transparency Version 2.0,” RFC Editor, Tech. Rep. RFC 9162, 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9162.html>
- [39] Google Chrome, “Certificate Transparency Policy,” 2026, accessed 2026-05-28. [Online]. Available: https://googlechrome.github.io/CertificateTransparency/ct_policy.html
- [40] National Institute of Standards and Technology, “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard,” 2024, accessed 2026-05-28. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [41] —, “FIPS 204: Module-Lattice-Based Digital Signature Standard,” 2024, accessed 2026-05-28. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [42] —, “FIPS 205: Stateless Hash-Based Digital Signature Standard,” 2024, accessed 2026-05-28. [Online]. Available: <https://csrc.nist.gov/pubs/fips/205/final>
- [43] PKI Consortium, “PKI Consortium,” 2026, practitioner and migration context; accessed 2026-05-28. [Online]. Available: <https://pkic.org/>
- [44] IETF, “CBOR Object Signing and Encryption (COSE): Initial Algorithms,” RFC Editor, Tech. Rep. RFC 9053, 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9053.html>
- [45] Coalition for Content Provenance and Authenticity, “C2PA Specifications,” 2026, accessed 2026-05-28. [Online]. Available: <https://spec.c2pa.org/specifications/>
- [46] Sigstore, “Sigstore Documentation,” 2026, accessed 2026-05-28. [Online]. Available: <https://docs.sigstore.dev/>

- [47] in-toto Project, "in-toto Attestation Framework," 2026, accessed 2026-05-28. [Online]. Available: <https://github.com/in-toto/attestation>
- [48] SPDX, "SPDX Specifications," 2026, accessed 2026-05-28. [Online]. Available: <https://spdx.dev/use/specifications/>
- [49] CycloneDX, "CycloneDX Specification Overview," 2026, accessed 2026-05-28. [Online]. Available: <https://cyclonedx.org/specification/overview>
- [50] World Wide Web Consortium, "Verifiable Credentials Data Model," 2026, recommendation entry point; accessed 2026-05-28. [Online]. Available: <https://www.w3.org/TR/vc-data-model/>
- [51] —, "Decentralized Identifiers (DIDs) v1.0," 2022, accessed 2026-05-28. [Online]. Available: <https://www.w3.org/TR/did-core/>
- [52] —, "Verifiable Credential Data Integrity," 2026, recommendation entry point; accessed 2026-05-28. [Online]. Available: <https://www.w3.org/TR/vc-data-integrity/>
- [53] United Nations Development Programme, "Digital Public Infrastructure," 2026, accessed 2026-05-28. [Online]. Available: <https://www.undp.org/digital/digital-public-infrastructure>
- [54] World Bank, "Digital Public Infrastructure and Services," 2026, accessed 2026-05-28. [Online]. Available: <https://www.worldbank.org/ext/en/topic/digital-and-ai/digital-public-infrastructure-and-services>
- [55] GovStack, "GovSpecs," 2026, accessed 2026-05-28. [Online]. Available: <https://govstack.global/our-offerings/govspecs/>
- [56] DPI Safeguards, "DPI Safeguards," 2026, accessed 2026-05-28. [Online]. Available: <https://www.dpi-safeguards.org/>
- [57] European Telecommunications Standards Institute, "ETSI EN 319 102-1 V1.4.1: Electronic Signatures and Infrastructures (ESI); Procedures for Creation and Validation of AdES Digital Signatures; Part 1: Creation and Validation," ETSI, Tech. Rep., 2024, accessed 2026-06-05. [Online]. Available: https://www.etsi.org/deliver/etsi_en/319100_319199/31910201/01.04.01_60/en_31910201v010401p.pdf
- [58] A. Sokolov, "X-Road/EATF AI-Agent Evidence Carrier: Reproducibility Package (v0.9)," 2026, dataset; version v0.9 under concept DOI 10.5281/zenodo.20468482; verified via Zenodo API 2026-06-09. [Online]. Available: <https://doi.org/10.5281/zenodo.20613101>

ANTON SOKOLOV is a researcher at Tyche Institute, Tallinn, Estonia. His work focuses on cryptographic evidence infrastructures for AI-system governance, eIDAS and PKI trust-service primitives, and reproducible audit patterns for agentic systems.

...